

A Hardware-Centric Federated Bulk-Synchronous Model for Heterogeneous Distributed Adaptive Systems

Author One, Author Two, and H. Sadoghi Yazdi

Abstract—Distributed adaptive systems are increasingly deployed on heterogeneous collections of processors, edge devices, sensor nodes, and networked machines whose computation rates, memory hierarchies, and communication links differ substantially. In such environments, the classical Bulk Synchronous Parallel (BSP) execution model provides an attractive abstraction because it separates local computation, communication, and barrier synchronization into analyzable supersteps. However, its global barrier becomes inefficient when fast nodes must repeatedly wait for slow nodes or for overloaded communication paths. This paper develops a hardware-centric view of a Federated Bulk Synchronous Parallel (FBSP) execution model for heterogeneous distributed adaptive learning systems. The proposed model first profiles the hardware behavior of nodes through an offline monitoring stage, then partitions the network into structurally and computationally coherent regions, and finally executes learning through two classes of supersteps: regional supersteps and global supersteps. In contrast to fully global BSP execution, FBSP constrains most communication and barrier synchronization to local regions while periodically invoking global synchronization to preserve network-wide consistency. The main contribution of this article is not a new stochastic optimizer, but a distributed hardware-system architecture that makes iterative adaptive learning more practical on heterogeneous platforms. The learning rule, based on stochastic-gradient-type updates, is treated as an internal computational kernel and summarized in the Appendix. Analytical cost expressions show how FBSP reduces idle time and communication volume by replacing repeated global barriers with region-aware synchronization. Simulation evidence on heterogeneous distributed networks indicates that increasing the interval between global supersteps can substantially reduce waiting time and communication load while retaining acceptable steady-state learning accuracy.

Index Terms—Bulk Synchronous Parallel, distributed systems, heterogeneous hardware, adaptive networks, barrier synchronization, straggler mitigation, edge learning, SimGrid, distributed adaptive filtering.

I. Introduction

Modern distributed adaptive systems are no longer executed on uniform clusters of identical processors. They frequently run over heterogeneous hardware composed of multi-core CPUs, embedded processors, mobile devices, sensor nodes, low-power edge devices, and machines connected through links of different bandwidth and latency. In such systems, the core bottleneck is not only the mathematical learning rule, but the interaction between

three hardware-level factors: local computation time, communication time, and synchronization delay.

Classical distributed learning methods often assume that participating nodes progress at comparable speeds. This assumption is fragile in real deployments. A slow processor, a congested link, or a node with a larger local workload can delay a global iteration. When a global barrier is imposed after every iteration, faster nodes remain idle until the slowest node completes its local computation and communication. This phenomenon, often called the straggler problem, reduces hardware utilization and weakens scalability.

The Bulk Synchronous Parallel (BSP) model is a natural starting point for disciplined distributed execution. A BSP program consists of supersteps, each containing local computation, communication, and a barrier synchronization. Its advantage is that runtime can be modeled explicitly through computation, communication, and synchronization costs. Its weakness in heterogeneous systems is that the global barrier is insensitive to node-level hardware diversity. In other words, BSP is analyzable and portable, but its unmodified form can be wasteful when hardware is heterogeneous.

This paper presents a hardware-centric article version of a Federated BSP (FBSP) model for heterogeneous distributed adaptive networks. The main idea is to preserve the analyzability of BSP while replacing the repeated global barrier with a two-level synchronization structure. Nodes are first profiled according to computational capability and then grouped into regions according to structural proximity, processing similarity, and trust or neighborhood weight. During online execution, most iterations use regional communication and regional barriers. A global superstep is invoked periodically to prevent regional models from drifting too far apart.

The article deliberately places stochastic gradient descent (SGD) in a secondary role. The SGD-type rule is treated as a local computational kernel executed inside each node. The main focus is the hardware and distributed-system layer: how nodes are profiled, how regions are formed, how barriers are scheduled, how communication volume is reduced, and how the runtime model changes compared with classical BSP.

II. Hardware and Distributed-System Motivation

A. Node Model

Consider a distributed adaptive system with N computing nodes. Each node v_i has local memory, a local processor, and communication links to a subset of other nodes. The hardware behavior of node v_i is characterized by

$$\mathcal{H}_i = (c_i, m_i, b_i, \ell_i),$$

where c_i denotes effective computation rate, m_i denotes available memory capacity or memory bandwidth, b_i denotes effective communication bandwidth to neighboring nodes, and ℓ_i denotes communication latency. In homogeneous clusters, these quantities may be approximately equal. In heterogeneous edge, cloud-edge, or sensor-network systems, they can differ significantly.

Each node stores a local data stream or local data partition. The system is data-parallel: every node performs local computation on its own data and periodically exchanges model parameters or intermediate states with neighbors. No permanent parameter server is assumed during online learning. This decentralized structure improves fault tolerance and avoids the single point of failure associated with centralized aggregation.

B. Why Classical BSP Becomes Inefficient

In a classical BSP execution, each superstep s has the form

local computation $\rightarrow$ communication $\rightarrow$ global barrier.

Let $w_i^{(s)}$ be the local computation time of node i during superstep s , $h_i^{(s)}$ the amount of communicated data, g the communication cost per data unit, and L the global barrier latency. The classical BSP cost of superstep s is

$$C_{\text{BSP}}^{(s)} = \max_i w_i^{(s)} + g \max_i h_i^{(s)} + L.$$

The total runtime over S supersteps is

$$C_{\text{BSP}} = \sum_{s=1}^S C_{\text{BSP}}^{(s)}.$$

This model is attractive because it is analyzable. However, the term $\max_i w_i^{(s)}$ means that every global iteration is controlled by the slowest node. Similarly, $g \max_i h_i^{(s)}$ means that the heaviest communication participant controls the communication phase. In heterogeneous hardware, these maxima can be much larger than the average node cost.

C. System-Level Objective

The objective of FBSP is to reduce hardware idle time and communication load while preserving enough global information exchange for stable adaptive learning. The proposed system therefore tries to satisfy three requirements:

- R1: reduce repeated waiting for globally slow nodes,
- R2: reduce unnecessary long-range communication,
- R3: keep periodic global consistency among regions.

The first two requirements are hardware-system goals; the third prevents the model from becoming purely local.

III. Federated BSP: Main System Architecture

A. Overview

The proposed FBSP architecture has two stages.

Offline stage: hardware-aware preparation. The system profiles node processing rates and communication behavior. A fixed-size calibration workload is assigned to nodes, and their processing time is measured. These measurements provide an estimate of computation capability. The network is then partitioned into regions using structural distance, computational similarity, and trust or neighborhood weights.

Online stage: two-level superstep execution. The adaptive learning process runs over the regioned network. Most iterations are regional supersteps, where nodes exchange information only with neighbors inside the same region and synchronize only regionally. Periodically, a global superstep is executed, allowing nodes to exchange information through the full network neighborhood and synchronize globally.

B. Network Representation

Let the distributed system be represented by a weighted graph

$$G = (V, E, A),$$

where $V = v_1, \dots, v_N$ is the set of computing nodes, E is the set of communication links, and $A = [a_{ij}]$ is a matrix of neighborhood or trust weights. A node v_i has a local neighbor set

$$\mathcal{N}_i = \{j : (v_i, v_j) \in E\} \cup \{i\}.$$

The local model parameter at node i and iteration t is denoted by $w_{i,t} \in \mathbb{R}^M$.

The system is partitioned into K regions:

$$\mathcal{R} = R_1, R_2, \dots, R_K, \quad R_r \subset V, \quad R_r \cap R_q = \emptyset \text{ for } r \neq q.$$

The regional neighbor set of node i is

$$\mathcal{N}_i^R = \mathcal{N}_i \cap R(i),$$

where $R(i)$ denotes the region containing node i . The global neighbor set remains $\mathcal{N}_i$.

C. Hardware-Aware Monitoring

The monitoring phase estimates the relative processing capability of each node. Let T_i^{cal} be the time required by node i to process a fixed calibration workload. A normalized compute score can be defined as

$$\rho_i = \frac{1/T_i^{\text{cal}}}{\max_j 1/T_j^{\text{cal}}}, \quad 0 < \rho_i \leq 1.$$

Thus, ρ_i close to one denotes a fast node, while smaller values indicate slower hardware.

The system may use ρ_i in two ways. First, it supports region formation by grouping nodes with similar computation rates. Second, it can guide workload assignment. For example, the local mini-batch size at node i may be selected as

$$B_i = B_{\min} + \lfloor \rho_i (B_{\max} - B_{\min}) \rfloor,$$

so faster nodes process larger batches while slower nodes process smaller ones. This helps align the per-superstep computation times and reduces barrier waiting.

D. Regioning by Structural and Hardware Similarity

A hardware-aware regioning method should avoid grouping very fast and very slow nodes into the same frequently synchronized group. At the same time, it should respect network structure and trust weights. One possible similarity score between nodes i and j is

$$S_{ij} = \alpha_s S_{ij}^{\text{struct}} + \alpha_c S_{ij}^{\text{comp}} + \alpha_t S_{ij}^{\text{trust}},$$

where

$$S_{ij}^{\text{struct}} = \exp\left(-\frac{d_{ij}^2}{\sigma_d^2}\right),$$

$$S_{ij}^{\text{comp}} = \exp\left(-\frac{|\rho_i - \rho_j|}{\sigma_c}\right),$$

and

$$S_{ij}^{\text{trust}} = a_{ij}.$$

Here, d_{ij} is the shortest-path distance or physical/network distance between nodes, and $\alpha_s, \alpha_c, \alpha_t \geq 0$ satisfy

$$\alpha_s + \alpha_c + \alpha_t = 1.$$

The purpose of S_{ij} is not merely graph clustering; it is hardware-aware graph partitioning. Regions should be structurally close, computationally coherent, and communication-feasible.

IV. FBSP Execution Model

A. Regional Superstep

A regional superstep is executed inside each region. For node i , the regional superstep consists of:

local update $\rightarrow$ regional exchange
 $\rightarrow$ regional combination $\rightarrow$ regional barrier.

Let $\psi_{i,t}$ denote the intermediate model computed locally at node i . In a regional superstep, node i updates

$$w_{i,t}^R = \sum_{j \in \mathcal{N}_i^R} a_{ji}^R \psi_{j,t},$$

where a_{ji}^R are normalized regional combination weights satisfying

$$a_{ji}^R \geq 0, \quad \sum_{j \in \mathcal{N}_i^R} a_{ji}^R = 1.$$

Algorithm 1 Hardware-Centric FBSP Execution

Require: Graph $G = (V, E, A)$, number of regions K ,
 global interval q , total iterations T

- 1: Offline monitoring:
- 2: for each node i do
- 3: assign a fixed calibration workload
- 4: measure T_i^{cal} and compute ρ_i
- 5: end for
- 6: build similarity matrix S using structural distance, compute score, and trust weight
- 7: partition G into regions $R_1, \dots, R_K$
- 8: assign local batch size B_i according to ρ_i
- 9: Online execution:
- 10: for $t = 1$ to T do
- 11: for each node i in parallel do
- 12: compute local intermediate model $\psi_{i,t}$
- 13: if $t \bmod q = 0$ then
- 14: exchange $\psi_{i,t}$ with $\mathcal{N}_i$
- 15: compute $w_{i,t}^G = \sum_{j \in \mathcal{N}_i} a_{ji}^G \psi_{j,t}$
- 16: else
- 17: exchange $\psi_{i,t}$ with $\mathcal{N}_i^R$
- 18: compute $w_{i,t}^R = \sum_{j \in \mathcal{N}_i^R} a_{ji}^R \psi_{j,t}$
- 19: end if
- 20: end for
- 21: if $t \bmod q = 0$ then
- 22: execute global barrier
- 23: else
- 24: execute regional barriers independently
- 25: end if
- 26: end for

B. Global Superstep

A global superstep is executed periodically. In this step, each node communicates with its full neighborhood:

$$w_{i,t}^G = \sum_{j \in \mathcal{N}_i} a_{ji}^G \psi_{j,t},$$

where

$$a_{ji}^G \geq 0, \quad \sum_{j \in \mathcal{N}_i} a_{ji}^G = 1.$$

The global superstep restores information flow across regions and limits long-term regional drift.

C. Global-Superstep Interval

Let q denote the global-superstep interval. If $q = 1$, every superstep is global and FBSP reduces to a BSP-like execution. If $q > 1$, then one global superstep is executed after $q - 1$ regional supersteps:

$$\underbrace{R, R, \dots, R}_{q-1 \text{ times}}, G, \underbrace{R, R, \dots, R}_{q-1 \text{ times}}, G, \dots$$

The parameter q is a system-level control knob. Larger q reduces communication and waiting time but increases the risk of regional model drift. Smaller q improves global consistency but increases synchronization cost.

V. Cost Model

A. Global BSP Cost

For a global BSP superstep,

$$C_G^{(s)} = \max_{i \in V} W_i^{(s)} + g_G \max_{i \in V} H_i^{G,(s)} + L_G,$$

where $W_i^{(s)}$ is local compute time, $H_i^{G,(s)}$ is global-neighborhood communication volume, g_G is the global communication coefficient, and L_G is the global barrier cost.

B. Regional FBSP Cost

For a regional superstep, each region advances with its own barrier. The cost is

$$C_R^{(s)} = \max_{r=1,\dots,K} \left[\max_{i \in R_r} W_i^{(s)} + g_r \max_{i \in R_r} H_i^{R,(s)} + L_r \right],$$

where $H_i^{R,(s)}$ is the regional communication volume and L_r is the regional barrier latency. Since $\mathcal{N}_i^R \subseteq \mathcal{N}_i$, typically

$$H_i^{R,(s)} \leq H_i^{G,(s)}.$$

C. Average FBSP Cost

With one global superstep every q iterations, the average cost per iteration is approximately

$$\bar{C}_{\text{FBSP}} = \frac{(q-1)C_R + C_G}{q}.$$

In contrast, classical BSP with global synchronization every iteration has

$$\bar{C}_{\text{BSP}} = C_G.$$

Thus, FBSP is advantageous when

$$C_R < C_G$$

and $q > 1$. The benefit increases when regional groups reduce both communication volume and idle time.

VI. Idle-Time Analysis

A. Global Waiting Time

Let

$$T_i^{(s)} = W_i^{(s)} + g H_i^{(s)}$$

denote the pre-barrier time of node i during superstep s . In a global barrier, the waiting time of node i is

$$I_i^{G,(s)} = \max_{j \in V} T_j^{(s)} - T_i^{(s)}.$$

The total global idle time is

$$I_G^{(s)} = \sum_{i \in V} \left(\max_{j \in V} T_j^{(s)} - T_i^{(s)} \right).$$

B. Regional Waiting Time

For regional synchronization, the waiting time of node $i \in R_r$ is

$$I_i^{R,(s)} = \max_{j \in R_r} T_j^{(s)} - T_i^{(s)}.$$

The total regional idle time is

$$I_R^{(s)} = \sum_{r=1}^K \sum_{i \in R_r} \left(\max_{j \in R_r} T_j^{(s)} - T_i^{(s)} \right).$$

Theorem 1 (Regional barriers do not increase total barrier waiting). For a fixed superstep s and a fixed partition $\mathcal{R}$ of V ,

$$I_R^{(s)} \leq I_G^{(s)}.$$

Proof. For every region R_r and every node $i \in R_r$,

$$\max_{j \in R_r} T_j^{(s)} \leq \max_{j \in V} T_j^{(s)}.$$

Therefore,

$$\max_{j \in R_r} T_j^{(s)} - T_i^{(s)} \leq \max_{j \in V} T_j^{(s)} - T_i^{(s)}.$$

Summing over all nodes in all regions gives

$$I_R^{(s)} \leq I_G^{(s)}.$$

□

Remark 1. The inequality is strict whenever at least one region does not contain a globally slowest node and contains at least one node that would otherwise wait for that globally slowest node.

VII. Communication Analysis

Let the model dimension be M . If every model exchange transfers M real values, the global communication volume per iteration is proportional to

$$V_G = M \sum_{i=1}^N |\mathcal{N}_i|.$$

The regional communication volume is

$$V_R = M \sum_{i=1}^N |\mathcal{N}_i^R|.$$

Since $\mathcal{N}_i^R \subseteq \mathcal{N}_i$,

$$V_R \leq V_G.$$

The average communication volume per iteration in FBSP is

$$\bar{V}_{\text{FBSP}} = \frac{(q-1)V_R + V_G}{q}.$$

Thus,

$$\bar{V}_{\text{FBSP}} \leq V_G,$$

with equality only when $q = 1$ or $V_R = V_G$.

Proposition 1 (Communication reduction). If $q > 1$ and at least one node has fewer regional neighbors than global neighbors, then

$$\bar{V}_{\text{FBSP}} < V_G.$$

Proof. If at least one node has $|\mathcal{N}_i^R| < |\mathcal{N}_i|$, then $V_R < V_G$. For $q > 1$,

$$\bar{V}_{\text{FBSP}} = \frac{(q-1)V_R + V_G}{q} < \frac{(q-1)V_G + V_G}{q} = V_G.$$

□

VIII. Staleness and Consistency

A regional-only system may drift because regions exchange no global information. FBSP prevents unbounded regional isolation by enforcing a global superstep every q iterations.

Let $\tau_i(t)$ denote the age of the last globally synchronized information used by node i at iteration t . Under a deterministic global interval q ,

$$0 \leq \tau_i(t) \leq q - 1.$$

Therefore, FBSP introduces bounded staleness:

$$\tau_{\max} \leq q - 1.$$

Lemma 1 (Bounded global staleness). If a global superstep is executed every q iterations, then no node uses globally unsynchronized information older than $q - 1$ regional iterations.

Proof. Immediately after a global superstep, $\tau_i(t) = 0$ for all nodes. Each regional step increases the age by one. Since at most $q - 1$ regional steps occur before the next global superstep, the maximum age is $q - 1$. □

Remark 2. The parameter q is therefore a hardware-system control parameter and a learning-consistency parameter at the same time. Increasing q reduces communication and waiting time but increases the maximum age of global information.

IX. Fault Tolerance and Portability

FBSP inherits useful systems properties from BSP. Because each superstep ends in a well-defined synchronization point, the state of the system is easier to inspect and checkpoint. Regional barriers provide even finer-grained checkpoint opportunities. A failure in one region can be isolated more naturally than in a fully global barrier system, especially when global synchronization is not required at every iteration.

The architecture is also portable. The FBSP abstraction does not require a specific processor type, memory layout, or interconnect technology. It can be implemented over MPI-like message passing, cluster simulation tools, edge networks, or hybrid cloud-edge systems. The key requirement is that each node can execute local computation, exchange model states with neighbors, and participate in regional or global barriers.

TABLE I
Representative FBSP behavior under different global-superstep intervals.

Approach	Avg. Comm./Iter.	Avg. Idle Time (s)	MSD (dB)
FBSP, $q = 1$	20.23	30.00	-19.27
FBSP, $q = 10$	14.60	19.60	-17.93
FBSP, $q = 50$	11.80	11.07	-16.69
FBSP, $q = 100$	10.04	8.83	-15.03

X. Evaluation Protocol

A. Simulation Environment

A hardware-centric evaluation should explicitly model computation rates and network links. A simulation platform such as SimGrid is suitable for this purpose because it supports heterogeneous processors, communication links, distributed applications, and scalable execution-time modeling. The evaluation should define:

node processing rates, link bandwidths,
link latencies, network topology,
local workload sizes.

B. Metrics

The following metrics are most relevant for evaluating the hardware-system contribution:

- Average idle time: the average waiting time caused by barriers.
- Average communication per iteration: the mean number of model exchanges or transmitted model vectors.
- Runtime per iteration: total simulated execution time divided by iteration count.
- Steady-state model error: a learning-quality metric such as mean-square deviation (MSD).
- Heterogeneity robustness: change in runtime and error when node speeds differ by factors such as $2\times$ or $10\times$.

The network-level MSD can be measured as

$$\text{MSD}^{\text{network}}(t) = \frac{1}{N} \sum_{i=1}^N \mathbb{E}|w^* - w_{i,t}|^2,$$

where w^* denotes the target parameter vector.

C. Representative Results

A representative comparison can be made by varying q in FBSP. When $q = 1$, all supersteps are global and the method behaves similarly to global BSP. Larger values of q increase the number of regional supersteps between global synchronizations.

The trend in Table I illustrates the system trade-off. Increasing q reduces communication and idle time. However, the steady-state error becomes less favorable because global information is exchanged less frequently. This is exactly the hardware-learning trade-off that FBSP exposes: the designer can tune q according to the desired balance between runtime efficiency and model accuracy.

TABLE II
Representative comparison under heterogeneous nonstationary settings.

Approach	Mixed Gaussian		Gaussian		Alpha-stable	
	Idle	MSD	Idle	MSD	Idle	MSD
FBSP, $q = 1$	30.00	-19.27	31.20	-42.25	30.53	-22.12
FBSP, $q = 50$	11.07	-16.69	11.30	-39.66	10.89	-20.86
RNA	9.88	-8.86	9.60	-24.44	10.00	-8.40
Synch-Switch	14.03	-12.50	14.60	-30.68	14.00	-14.30
PSP	10.65	-13.71	10.72	-34.45	11.20	-17.06

Table II shows the role of FBSP as a systems compromise. The fully global case, $q = 1$, obtains strong accuracy but suffers high idle time. The regional-global case, $q = 50$, keeps idle time close to competing low-wait methods while preserving substantially better error behavior than purely asynchronous or weakly synchronized alternatives.

XI. Discussion

The essential contribution of FBSP is the separation between the learning kernel and the execution substrate. Many distributed learning papers focus on the optimizer. In contrast, FBSP asks how an iterative adaptive computation should be executed on heterogeneous hardware. This distinction matters because a mathematically strong optimizer may still underperform if every iteration is slowed down by global barriers and excessive communication.

FBSP is most suitable when:

- the network is heterogeneous,
 - the learning process is iterative,
 - nodes have local data,
 - communication is expensive,
 - full global synchronization at every iteration is unnecessary.
- It is less appropriate for very sparse graphs where regional communication cannot preserve enough information flow, or for systems where all nodes are already homogeneous and communication is inexpensive.

XII. Conclusion

This article presented a hardware-centric formulation of Federated BSP for heterogeneous distributed adaptive systems. The central idea is to retain the analyzable superstep structure of BSP while replacing repeated global barriers with a two-level regional-global synchronization mechanism. The offline monitoring stage estimates node processing capability; the regioning stage groups nodes according to structural and computational similarity; and the online execution stage alternates between regional and global supersteps. The resulting model reduces barrier waiting and communication volume while bounding global staleness through the interval parameter q .

The key message is that the main novelty lies not in SGD itself, but in the distributed execution architecture surrounding the learning kernel. By treating computation

rate, communication volume, barrier waiting, and synchronization scope as first-class design variables, FBSP provides a practical system model for adaptive learning over heterogeneous hardware.

Appendix A

SGD-Type Local Learning Kernel

This appendix summarizes the stochastic-gradient-type kernel used inside the proposed distributed system. The main paper treats this kernel as a replaceable local computation module.

Let node i receive local input vector $u_{i,t} \in \mathbb{R}^M$ and desired scalar output $d_i(t)$. A standard linear observation model is

$$d_i(t) = u_{i,t}^T w^* + \nu_i(t),$$

where w^* is the unknown parameter vector and $\nu_i(t)$ is measurement noise.

The local prediction error is

$$e_i(t) = d_i(t) - u_{i,t}^T w_{i,t-1}.$$

A generic regularized local objective can be written as

$$J_i(w) = \mathbb{E}[\phi(e_i(t))] + \frac{\lambda}{2}|w|^2,$$

where $\phi(\cdot)$ is a loss function and $\lambda \geq 0$ is a regularization coefficient.

A scaled stochastic-gradient update has the form

$$\psi_{i,t} = w_{i,t-1} \mu_i(t) S_t^{-1} \left(\lambda w_{i,t-1} - \widehat{\nabla J_i}(w_{i,t-1}) \right),$$

where $\mu_i(t)$ is a possibly node-dependent step size, S_t is a positive-definite scaling matrix, and $\widehat{\nabla J_i}$ is a stochastic gradient estimate.

For squared loss,

$$\phi(e) = \frac{1}{2}e^2, \quad \phi'(e) = e.$$

For robust learning under non-Gaussian noise, a pseudo-Huber-type loss may be used:

$$\phi_\delta(e) = \delta^2 \left(\sqrt{1 + \left(\frac{e}{\delta}\right)^2} - 1 \right),$$

with derivative

$$\phi'_\delta(e) = \frac{e}{\sqrt{1 + \left(\frac{e}{\delta}\right)^2}}.$$

The corresponding robust stochastic-gradient term is

$$\widehat{\nabla J_i}(w_{i,t-1}) = \frac{e_i(t) u_{i,t}}{\sqrt{1 + \left(\frac{e_i(t)}{\delta}\right)^2}}.$$

After the local intermediate update $\psi_{i,t}$, FBSP determines whether node i performs regional or global combination:

$$w_{i,t} = \begin{cases} \sum_{j \in \mathcal{N}_i^R} a_{ji}^R \psi_{j,t}, & \text{regional superstep,} \\ \sum_{j \in \mathcal{N}_i} a_{ji}^G \psi_{j,t}, & \text{global superstep.} \end{cases}$$

Thus, SGD is the local numerical engine, while FBSP is the hardware-aware distributed execution model that decides where, when, and with whom each node communicates.

Appendix B

Practical Implementation Notes

A practical implementation of FBSP should include:

- a profiling routine to estimate T_i^{cal} and ρ_i ;
- a graph partitioning module for hardware-aware re-gioning;
- separate communicators for regional and global message exchange;
- a scheduler implementing the q -controlled regional/global superstep pattern;
- checkpointing at regional and global barriers;
- logging of computation time, communication time, idle time, and model error.

In an MPI-based implementation, each region can be mapped to a regional communicator, while the full network uses a global communicator. In an edge deployment, regional barriers can be mapped to local gateway-level synchronization, while global barriers can be performed through less frequent inter-region coordination.

References

- [1] L. G. Valiant, “A bridging model for parallel computation,” *Communications of the ACM*, vol. 33, no. 8, pp. 103–111, 1990.
- [2] A. H. Sayed, “Adaptation, learning, and optimization over networks,” *Foundations and Trends in Machine Learning*, vol. 7, no. 4–5, pp. 311–801, 2014.
- [3] J. Dean and S. Ghemawat, “MapReduce: simplified data processing on large clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [4] D. Culler, R. Karp, D. Patterson, A. Sahay, K. Schauser, E. Santos, R. Subramonian, and T. von Eicken, “LogP: towards a realistic model of parallel computation,” *ACM SIGPLAN Notices*, vol. 28, no. 7, pp. 1–12, 1993.
- [5] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, “Versatile, scalable, and accurate simulation of distributed applications and platforms,” *Journal of Parallel and Distributed Computing*, vol. 74, no. 10, pp. 2899–2917, 2014.
- [6] B. Recht, C. Re, S. Wright, and F. Niu, “Hogwild!: A lock-free approach to parallelizing stochastic gradient descent,” in *Advances in Neural Information Processing Systems*, 2011.
- [7] X. Lian, C. Zhang, H. Zhang, C. Hsieh, W. Zhang, and J. Liu, “Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent,” in *Advances in Neural Information Processing Systems*, 2017.
- [8] R. H. Bisseling, *Parallel Scientific Computation: A Structured Approach using BSP*. Oxford University Press, 2020.